

Multi-Client Data Isolation

with Pinecone Metadata Filtering

1000 Docs · 10 Clients · One Index · Zero Data Leakage

Grounded in your `pinecone_upsert_custom_metadata.py` &
`pinecone_query_with_without_filter.py` patterns

Created: June 28, 2026 | 06:04



Table of Contents

1. The Real Use Case: 1000 Docs, 10 Clients	3
2. Why a Single Index Without Filtering is Dangerous	3
3. How Metadata Filtering Creates Client Isolation	4
4. Metadata Schema Design	5
5. Isolation Strategies Compared	6
6. Your Existing Code — What You Already Have	6
7. Extending Your Code for 10 Clients	7
8. Enforcing Filters Server-Side (The Critical Rule)	9
9. Pinecone Filter Operators Reference	10
10. Advanced Patterns: Namespace + Filter Hybrid	10
11. Deleting a Client's Data	11
12. Common Security Pitfalls	12
13. References	13

1. The Real Use Case: 1000 Docs, 10 Clients

Imagine you are building a RAG-based document assistant as a SaaS product. You have **10 clients**, each with roughly 100 documents — contracts, HR policies, financial reports, technical manuals — totalling **1000 documents** and approximately **100,000 chunks** in your Pinecone index.

The core challenge: **all 100,000 chunks live in the same vector index**. When an Acme employee asks a question, the semantic search must only surface Acme's chunks — never Globex's contracts or Initech's HR policies. This is not just a product feature; it is a **legal and contractual obligation**.

#	client_id	Docs	Chunks (~100/doc)	Category Tags
1	acme	120	~12,000	legal, hr, finance
2	globex	80	~8,000	contracts, sales
3	initech	95	~9,500	hr, policy
4	umbrella	110	~11,000	research, safety
5	hooli	90	~9,000	tech, product
6	dunder	100	~10,000	hr, finance, ops
7	vandelay	85	~8,500	import, export
8	bluth	105	~10,500	legal, real-estate
9	piedpiper	75	~7,500	tech, patents
10	massdyn	140	~14,000	research, bio, legal
—	TOTAL	1,000	~100,000	All in one index

Table 1 — Scale breakdown: 10 clients × ~100 docs × ~100 chunks/doc = 100K vectors in one index

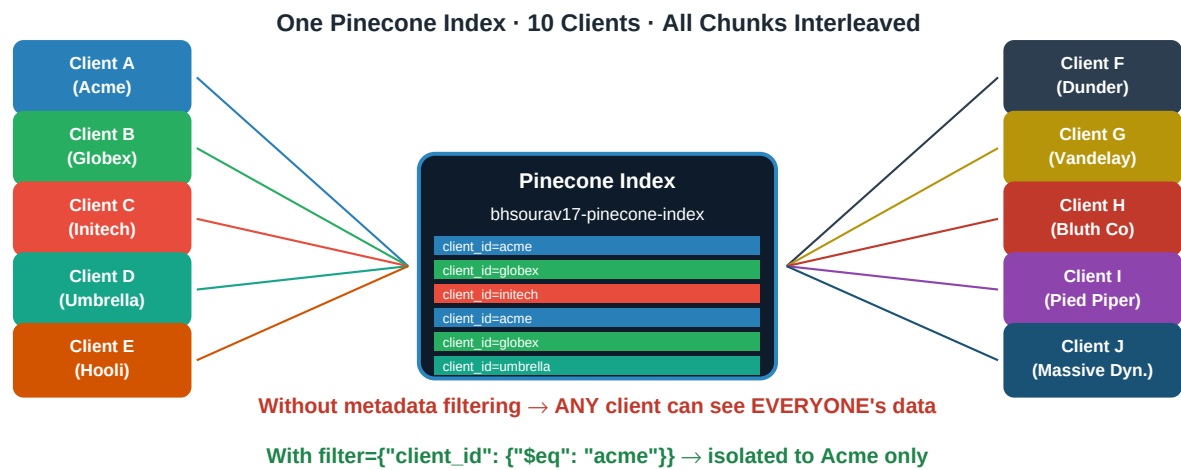
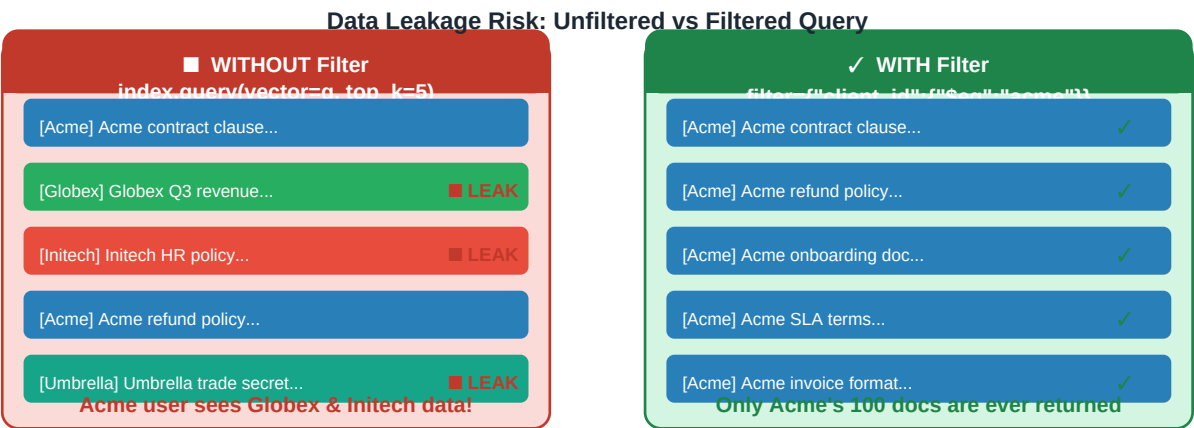


Figure 1 — All 10 clients share one Pinecone index; metadata filtering is the only boundary

2. Why a Single Index Without Filtering is Dangerous

Pinecone's ANN (approximate nearest neighbour) search is **globally blind to ownership**. Without a filter, a query from an Acme user will return the top-K most semantically similar chunks — from *any* client whose documents happen to be relevant.

Consider this query from an Acme employee: "What is our refund window?" — without a filter, chunks from Globex's return policy, Initech's SLA terms, and Umbrella's customer agreement could all rank higher than Acme's own document, because they are semantically similar text.



★ **Key Concept** The filter is mathematically equivalent to a WHERE clause in SQL.
`index.query(vector=q, filter={'client_id':{'$eq':'acme'}})` means: 'find the top-K most similar vectors, but only among those where `client_id = acme`'.

4. Metadata Schema Design

Every chunk upserted to Pinecone carries a metadata dictionary. Designing this schema carefully at the start prevents painful re-indexing later. The table below shows the recommended schema, extending the fields you already use in `pinecone_upsert_custom_metadata.py`:

Field	Type	Example (Acme doc)	Purpose
<code>client_id</code>	str	"acme"	PRIMARY isolation key — always filter on it
<code>source_filename</code>	str	"acme_refund_policy.pdf"	Secondary filter by document
<code>page_number</code>	int	3	Pagination, citation
<code>start_index</code>	int	450	Char offset — matches your <code>add_start_index</code>
<code>category_label</code>	str	"legal"	Topic filter within a client
<code>doc_id</code>	str	"acme-doc-007"	Unique doc ID for delete/update
<code>text</code>	str	"Our refund window is 30 days..."	Chunk content for display

client_id is the mandatory isolation field — every other field is optional enrichment
Figure 4 — Recommended metadata schema; fields in bold are mandatory for isolation

4.1 The `client_id` Field is the Isolation Anchor

Every single query in a multi-client system must include `filter={'client_id': {'$eq': client_id}}`. This one field is your data boundary. All other metadata fields (`category_label`, `doc_id`, `page_number`) provide additional filtering *within* a client's data — they never replace `client_id`.

■ Your existing code uses `source_filename` as a filter — which works for your current single-user setup. For multi-client, `client_id` must be added as the primary isolation key. `source_filename` becomes a secondary filter.

5. Isolation Strategies Compared

Strategy	How It Works	Pros	Cons	Use When
Metadata Filter (<code>client_id</code>) ★ Recommended	Tag every chunk with <code>client_id</code> ; filter every query	Simple setup One index Low cost Flexible filters	No hard isolation at infra level Dev discipline needed	Most RAG apps; your current setup with 10 clients
Namespaces	Pinecone namespace per client; queries scoped to namespace	Logical separation Fast namespace scoped queries	Can't filter across namespaces easily All in one index	When you want logical separation without extra cost
Separate Index per Client	Each client gets their own Pinecone index	Strongest isolation Independent scaling Easy delete per client	Expensive at scale (1 index per client = 10 paid indexes)	High-security / enterprise clients with strict compliance
Hybrid (Namespace + Metadata Filter)	Namespace per client + metadata filter for sub-categories	Two layers of isolation Flexible sub-queries	Most complex setup Higher dev overhead	Large-scale SaaS with compliance requirements

Table 2 — Four isolation strategies: metadata filter is the recommended starting point

6. Your Existing Code — What You Already Have

Your Pinecone code is already 80% of the way there. Here is what each file contributes and what needs to change:

6.1 pinecone_upsert_custom_metadata.py — metadata shape is right

✓ **Your Code Already Does This** You already store `source_filename` and `category_label` in metadata. Adding `client_id` is a one-line change.

```
# Your current metadata (pinecone_upsert_custom_metadata.py, line 51-58)
"metadata": {
    "text":          chunk.page_content,
    "source":        chunk.metadata.get("source", ""),
    "page":          chunk.metadata.get("page", 0),
    "start_index":   chunk.metadata.get("start_index", 0),
    "source_filename": os.path.basename(chunk.metadata.get("source", "")),
    "page_number":    chunk.metadata.get("page", 0) + 1,
    "category_label": CATEGORY,
    # ↑ Add one more field: ↓
    "client_id":      CLIENT_ID,    # e.g. "acme"
}
```

6.2 pinecone_query_with_without_filter.py — filter pattern is right

✓ **Your Code Already Does This** Your `query_filtered()` function already demonstrates the exact filter pattern needed. Swap `source_filename` for `client_id` as the primary isolation key.

```
# Your current filter (pinecone_query_with_without_filter.py, line 37-44)
def query_filtered(query_vector, source_filename, top_k=TOP_K):
    results = index.query(
        vector=query_vector,
        top_k=top_k,
        filter={"source_filename": {"$eq": source_filename}}, # ← current
        include_metadata=True,
    )
    return results.matches

# What it becomes in multi-client:
def query_for_client(query_vector, client_id, top_k=TOP_K):
    results = index.query(
        vector=query_vector,
        top_k=top_k,
        filter={"client_id": {"$eq": client_id}},           # ← client isolation
        include_metadata=True,
    )
    return results.matches
```

6.3 pinecone_search_on_metadata.py — production pattern

✓ **Your Code Already Does This** Your `search()` function already returns `Match` objects with `.score`, `.id`, `.metadata` — exactly the right shape for a production query layer.

7. Extending Your Code for 10 Clients

7.1 Multi-Client Upsert

Extend your upsert pipeline to accept a `client_id` parameter and tag every chunk with it:

```
import os
from dotenv import load_dotenv
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from openai import OpenAI
from pinecone import Pinecone

load_dotenv()

INDEX_NAME = "bhsourav17-pinecone-index"
BATCH_SIZE = 100

openai_client = OpenAI()
pc = Pinecone()
index = pc.Index(INDEX_NAME)

def get_embeddings(texts: list[str]) -> list[list[float]]:
    response = openai_client.embeddings.create(
        input=texts, model="text-embedding-3-small",
    )
    return [item.embedding for item in response.data]

def upsert_client_docs(
    client_id: str,          # e.g. "acme"
    pdf_paths: list[str],   # list of PDF file paths for this client
    category: str = "general",
    chunk_size: int = 200,
    chunk_overlap: int = 50,
) -> int:
    """Load, chunk, embed, and upsert documents for ONE client."""
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=chunk_size,
        chunk_overlap=chunk_overlap,
        add_start_index=True,
    )
    all_chunks = []
    for pdf_path in pdf_paths:
        loader = PyPDFLoader(pdf_path)
        pages = loader.load()
        all_chunks.extend(splitter.split_documents(pages))

    vectors = []
    for i, chunk in enumerate(all_chunks):
        vectors.append({
            "id": f"{client_id}-chunk-{i}",  # client-scoped ID prevents collisions
            "values": None,
            "metadata": {
                "client_id": client_id,  # ← PRIMARY isolation key
                "text": chunk.page_content,
                "source_filename": os.path.basename(chunk.metadata.get("source", "")),
                "page_number": chunk.metadata.get("page", 0) + 1,
                "start_index": chunk.metadata.get("start_index", 0),
                "category_label": category,
                "doc_id": f"{client_id}-{os.path.basename(chunk.metadata.get('source', ''))}"
            }
        })
    return len(vectors)
```


7.2 Multi-Client Query

[illegible]

8. Enforcing Filters Server-Side (The Critical Rule)

The most important security rule in multi-tenant RAG: **never trust client_id from the user**. It must always be resolved from the authenticated session on the server side.

■ **Security Warning** If your API accepts `client_id` as a query parameter or in the request body, any user can change it to 'acme' or 'competitor-corp' and read their data. The metadata filter is only as strong as where `client_id` comes from.

8.1 Correct Pattern: FastAPI with JWT

[illegible]

9. Pinecone Filter Operators Reference

Pinecone supports MongoDB-style filter operators. All of these compose with `client_id` as the base filter:

Operator	Meaning	Example Filter
\$eq	Equals	<code>{"client_id": {"\$eq": "acme"}}</code>
\$ne	Not equals	<code>{"client_id": {"\$ne": "acme"}}</code>
\$in	In a list of values	<code>{"client_id": {"\$in": ["acme", "globex"]}}</code>
\$nin	Not in a list	<code>{"client_id": {"\$nin": ["acme"]}}</code>
\$and	All conditions true	<code>{"\$and": [{"client_id": {"\$eq": "acme"}}, {"category": {"\$eq": "legal"}}]}</code>
\$or	Any condition true	<code>{"\$or": [{"category": {"\$eq": "legal"}}, {"category": {"\$eq": "hr"}}]}</code>
\$gt	Greater than (nums)	<code>{"page_number": {"\$gt": 5}}</code>
\$gte	Greater or equal	<code>{"page_number": {"\$gte": 1}}</code>

Table 3 — Pinecone metadata filter operators with examples

■ For multi-client systems, always wrap additional filters inside `$and` with `client_id` as the first condition. This guarantees isolation is never accidentally dropped when adding sub-filters.

10. Advanced Pattern: Namespace + Filter Hybrid

For enterprise-grade isolation, combine Pinecone **namespaces** (one per client) with **metadata filters** (for sub-categories within a client). This gives two independent layers of isolation.

```
# Upsert into a client-specific namespace
index.upsert(
    vectors=batch,
    namespace=client_id,  # e.g. "acme"
)

# Query within the client's namespace (no cross-namespace bleed possible)
results = index.query(
    vector=query_vector,
    top_k=5,
    namespace=client_id,  # namespace isolation
    filter={"category_label": {"$eq": "legal"}},  # sub-filter within client
    include_metadata=True,
)

# Note: client_id in metadata is still useful for audit logging
# even when using namespaces, so keep it in your metadata schema.
```

■ Namespaces in Pinecone are logical partitions within an index — queries can only reach vectors in the specified namespace. This is stronger than metadata filtering alone because namespace enforcement is at the infrastructure level, not application logic.

11. Deleting a Client's Data

When a client offboards, you need to delete *all their chunks* without touching other clients' data. Your existing `pinecone_delete.py` uses `delete(delete_all=True)` which wipes the entire index — dangerous in a multi-client setup.

```
# ✗ Your current pinecone_delete.py – WIPES EVERYTHING
index.delete(delete_all=True)    # DO NOT use in multi-client setup

# ✓ Delete only ONE client's chunks using their client-scoped IDs
# Option A: if you used client-prefixed IDs (e.g. "acme-chunk-0", "acme-chunk-1")
def delete_client_by_ids(client_id: str, total_chunks: int):
    ids_to_delete = [f"{client_id}-chunk-{i}" for i in range(total_chunks)]
    # Delete in batches of 1000 (Pinecone limit)
    for start in range(0, len(ids_to_delete), 1000):
        batch = ids_to_delete[start:start+1000]
        index.delete(ids=batch)
    print(f"Deleted {total_chunks} chunks for client: {client_id}")

# Option B: if using namespaces (cleanest – one call)
def delete_client_namespace(client_id: str):
    index.delete(delete_all=True, namespace=client_id)
    print(f"Namespace '{client_id}' deleted entirely")

# Option C: fetch IDs by metadata filter, then delete
# (use when IDs are not client-prefixed and no namespace)
def delete_client_by_metadata(client_id: str, total_to_delete: int = 10_000):
    """Paginate through client's vectors and delete them in batches."""
    deleted = 0
    while deleted < total_to_delete:
        # Pinecone list() paginates vector IDs (no embedding needed)
        results = index.query(
            vector=[0.0] * 1536,      # dummy vector
            top_k=1000,
            filter={"client_id": {"$eq": client_id}},
            include_values=False,
            include_metadata=False,
        )
        ids = [m.id for m in results.matches]
        if not ids:
            break
        index.delete(ids=ids)
        deleted += len(ids)
    print(f"Deleted {deleted} chunks for client: {client_id}")
```

■ Use client-prefixed IDs (e.g. 'acme-chunk-42') from the start — it makes targeted deletion trivial and gives you a free audit trail of which chunks belong to which client.

12. Common Security Pitfalls

■ <i>client_id</i> trusted from user input	NEVER read <code>client_id</code> from the request body or query params. Always resolve it from a verified JWT or server-side session. A user can trivially change a query param to access another client's data.
■ <i>Forgetting the filter on one code path</i>	If you have 10 API endpoints and 9 of them filter by <code>client_id</code> , but one doesn't — that one is a data breach waiting to happen. Create a single <code>query_for_client()</code> wrapper and use it everywhere.
■ <i>Using <code>delete(delete_all=True)</code> in a shared index</i>	Your current <code>pinecone_delete.py</code> wipes the entire index. In a multi-client setup this destroys all 10 clients' data. Use client-prefixed IDs or namespaces for safe per-client deletion.
■ <i>No audit logging on queries</i>	Without logs of who queried what and which chunks were returned, you cannot prove isolation in a compliance audit. Log: timestamp, <code>client_id</code> , query hash, top-1 chunk ID for every query.
■ <i>Not testing cross-client leakage</i>	Write an integration test that upserts docs for 2 clients, then queries as client A and asserts that no client B chunks appear in results. This test must run in CI on every deploy.
■ <i>Metadata filter not on indexed fields</i>	Pinecone requires metadata fields used in filters to be included in the index's metadata config (for pod-based indexes). For serverless indexes, all metadata is filterable by default.

12.1 Implementation Checklist

- ■ Every chunk upserted has `client_id` in metadata
- ■ Vector IDs use client prefix: `f'{client_id}-chunk-{i}'`
- ■ Every query goes through a single `query_for_client()` wrapper
- ■ `client_id` is resolved from JWT — never from request body
- ■ `delete(delete_all=True)` is removed or gated in multi-client code
- ■ Integration test verifies zero cross-client results
- ■ Query logs include `client_id` and timestamp
- ■ `pinecone_delete.py` has been updated to use per-client deletion

13. References

- [1] Pinecone Docs — Metadata Filtering <https://docs.pinecone.io/guides/data/filter-with-metadata>
- [2] Pinecone Docs — Namespaces <https://docs.pinecone.io/guides/indexes/use-namespaces>
- [3] Pinecone Docs — `index.query()` API Reference <https://docs.pinecone.io/reference/api/data-plane/query>
- [4] Pinecone Docs — `index.delete()` API Reference <https://docs.pinecone.io/reference/api/data-plane/delete>
- [5] Pinecone Docs — Serverless vs Pod-based indexes <https://docs.pinecone.io/guides/indexes/choose-a-pod-type-and-size>
- [6] OpenAI — text-embedding-3-small <https://platform.openai.com/docs/guides/embeddings>
- [7] LangChain — RecursiveCharacterTextSplitter https://python.langchain.com/docs/how_to/recursive_text_splitter/
- [8] FastAPI — Security with JWT <https://fastapi.tiangolo.com/tutorial/security/oauth2-jwt/>

Your code files referenced in this document:

[pinecone_upsert.py](#) – basic single-doc upsert pipeline
[pinecone_upsert_custom_metadata.py](#) – multi-doc upsert with `category_label`
[pinecone_query_with_without_filter.py](#) – filter comparison and `Precision@K`
[pinecone_search_on_metadata.py](#) – `source_filename` filtered search
[pinecone_similarity_queries.py](#) – unfiltered similarity queries
[pinecone_delete.py](#) – delete all vectors (caution in multi-client)

Generated on June 28, 2026 at 06:04. Based on your actual Pinecone code in `AI_study/learning/codes/`. Pinecone SDK v3+, Python 3.12.